

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Short Answer Test

Weightage: 10%

QUESTIONS:

Total Marks: 20

Question 1 (5 Marks)

Suppose that a company wants its website to work seamlessly on mobile, tablet, and desktop devices. Explain the concept of responsive web design and its importance. Describe the key CSS techniques used to achieve responsiveness, and explain how CSS Flexbox or Grid can be used to design adaptive layouts. Illustrate your answer with a simple example layout and suggest improvements to enhance user experience across different devices.

Question 2 (5 Marks)

Suppose that a webpage contains a form where user input must be validated before submission. Explain the concept of client-side validation and its purpose. Describe how JavaScript event handling is used in validation and explain the role of regular expressions in validating inputs such as email or phone numbers. Provide a suitable example and suggest improvements for better usability and security.

Question 3 (5 Marks)

Suppose that a webpage layout breaks when the browser window is resized. Explain the CSS box model and its components, and describe how CSS positioning techniques affect layout structure. Identify possible causes of layout breaking and explain how media queries can be used to resolve these issues. Suggest best practices to ensure a stable and responsive layout.

Question 4 (5 Marks)

Suppose that a website requires dynamic content updates without reloading the entire page. Explain the concept of the Document Object Model (DOM) and its role in web development. Describe how JavaScript can be used to manipulate the DOM for updating webpage content dynamically. Provide an example and suggest improvements to enhance performance and user experience.

Code of Conduct:

I Ebin Antony P (192373011) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application	20	Correctly	Applies	Limited or	No or

of Knowledge		applies concepts to solve the question/problem	concepts with minor mistakes	partially correct application	incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1. Responsive Web Design.

Introduction:

Responsive Web Design (RWD) is a web development approach that makes a website automatically adjust its layout, images, text, and other elements according to the screen size and device.

Importance of Responsive Web Design

- Provides a better user experience on all devices.
- Avoids the need to create separate websites for mobile and desktop.
- Improves readability and navigation.
- Reduces horizontal scrolling and unnecessary zooming.
- Improves accessibility and search-engine ranking.

Key CSS Techniques

- * Fluid layouts - Use percentages, rem, em, vw instead of fixed sizes.
- * Flexible images - Use max-width: 100% so images fit within containers.
- * Media Queries - Apply different CSS rules for different screen sizes.
- * Flexbox - Arranges elements in rows or columns.
- * CSS Grid - Create responsive two-dimensional layout using rows & columns.

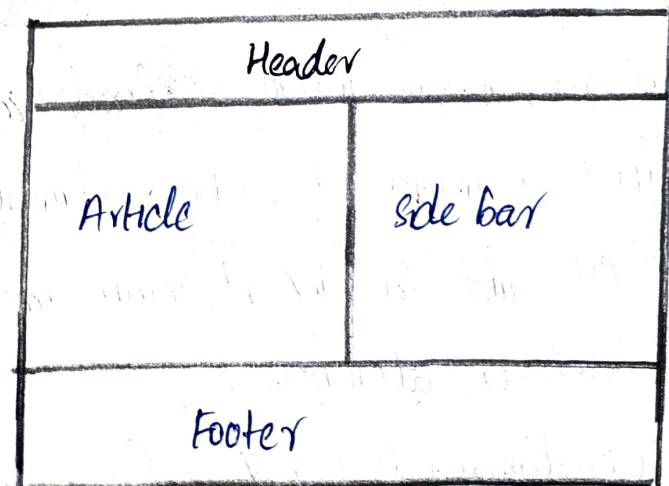
Example: CSS Flexbox + Media Query:

```
.container {  
  display: flex;  
  gap: 20px; }  
.box {  
  width: 50%; }  
@ media (max-width: 600px) {  
  .container {  
    flex-direction: column; }  
  .box { width: 100%; }  
}
```

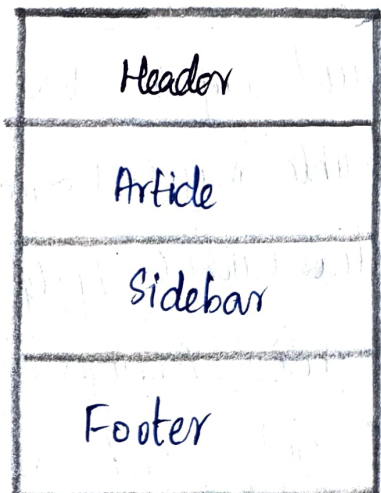
HTML:

```
<div class="container">  
  <div class="box">Article</div>  
  <div class="box">Sidebar</div>  
</div>
```

Layout (Desktop)



Layout (Mobile)



Improvements:

- Use readable font sizes and sufficient spacing
- Make buttons large enough for touch interaction.
- Optimize images for faster loading.
- Use mobile-friendly navigation
- Test the website on different screen sizes and browsers.

2. Client-Side Validation

Introduction:

Client-side validation is process of checking user-entered data in the browser. before sending from to the server. JavaScript is commonly used to validate ~~form~~^{form} fields.

Purpose of Client-Side Validation

- Prevents incorrect or incomplete data
- Gives immediate feedback to the user.
- Reduces unnecessary requests to the server.
- Improves user experience.
- Helps ensure that required fields contain valid information.

Java Script Event Handling

Uses events such as:

- submit - occurs when the form is submitted
- input - occurs when user changes a field
- blur - occurs when a field loses focus.

Role of Regular Expressions

A regular expression (Regex) is a pattern used to check whether input follows a particular format.

For eg:

- Email: tomholland@gmail.com
- Phone: 9121367580

JavaScript validation with Regex:

```
function validate () {  
    let email = document.getElementById("email").value;  
    let pattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;  
    if (!pattern.test(email)) {  
        alert("Enter a valid email");  
        return false; }  
    return true; }
```

HTML:

```
<input type="text" id="email">  
<button onclick="validate()"> Submit </button>
```

Phone Regex eg:

```
let phone pattern = /^[0-9]{10}$/;
```

Improvements

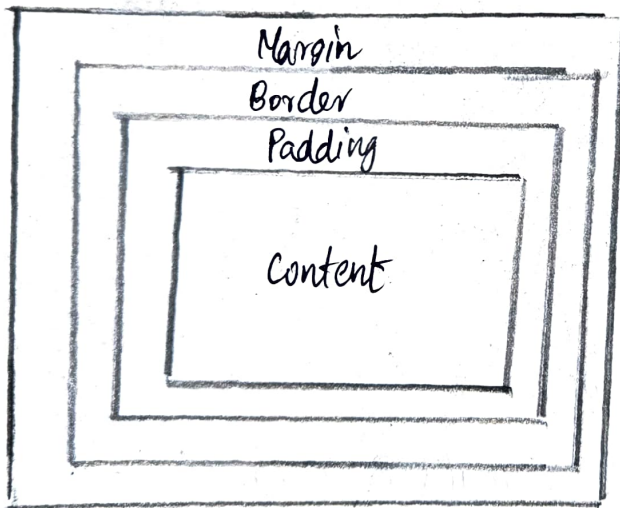
- Display clear error messages near the relevant fields.
- Validate input while user is entering data when appropriate.

- Use HTML 5 attributes such as `required`, `type="email"`, `pattern`.
- Use server-side validation as well.
- Never depend only on client-side validation for security.
- Sanitize and validate data on the server before ~~starting~~ storing or processing it.

3. CSS Box Model, Positioning and Media Queries

CSS Box Model

The CSS Box Model describes how every HTML element is represented as a rectangular box.



- Content - actual text, image or other information
- Padding - space between content and border.
- Border - line surrounding the padding and content
- Margin - space outside the border

These are the main four components of CSS Box Model

CSS Positioning

CSS provides different positioning techniques:

- Static - default position
- Relative - element is positioned relative to its normal position
- Absolute - positioned relative to its nearest positioned ancestor
- Fixed - remains fixed relative to the browser window.
- Sticky - behaves like relative until a scrolling position is reached.

Improper use of absolute or fixed positioning can cause elements to overlap or move outside the screen.

Causes of Layout Breaking

- Fixed widths and heights
- Excessive margins and padding.
- Large images or texts.
- Incorrect use of position: absolute.
- Elements that cannot shrink on smaller screens.
- Missing Media Queries.
- Incorrect box-sizing calculations.

CSS Box Model & Media Queries

```
• box { width: 300px;  
padding: 20px;  
border: 2px solid black;
```

④

ebin.

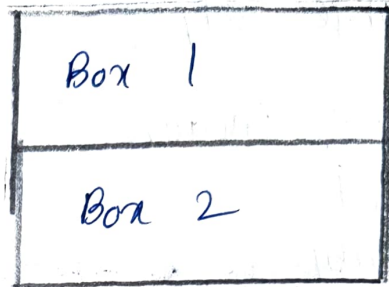
margin: 10 px;
box-sizing: border-box; }

Media Query
@ media (max-width: 600px) {
 box { width: 100%;
 }
}

On desktop boxes appear
side by side



On a mobile screen



Best Practices

- Use box-sizing: border-box
- Prefer flexible units such as %, rem, vw and fr.
- Avoid unnecessary fixed widths & heights
- Use flexbox or grid for layouts.
- Use media queries for smaller screens
- Make images responsive using max-width: 100 %.
- Test layouts at different screen sizes.

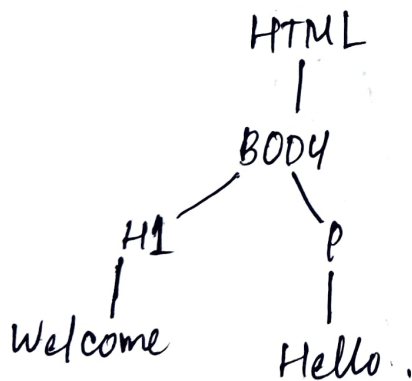
4. DOM and Dynamic Web Content

The Document Object Model (DOM) is a programming interface that represents an HTML document as a tree of objects or nodes. JavaScript can access and modify these objects to change webpage content dynamically.

Eg DOM structure

```
<html>  
  <body>  
    <h1> Welcome </h1>  
    <p> Hello </p>  
  </body> </html>
```

The DOM can be represented as:



Role of DOM

DOM allows JavaScript to:

- Change HTML content
- Change CSS styles
- Add or remove HTML elements

- Change attributes
- Respond to user actions
- Dynamically update webpage content without reloading the complete page

DOM Manipulation.

HTML:

```
<h2 id = "title"> welcome </h2>  
<button onclick = "changeText()"> Click </button>
```

Javascript:

```
function changeText() {  
    document.getElementById("title").textContent = "Content Updated!";  
}
```

Other useful DOM methods:

```
document.querySelector("u.box");  
document.createElement("u.p");  
element.appendChild(newElement);  
element.remove();
```

Improvements for Performance

- Update only the required DOM elements
- Avoid unnecessary DOM Manipulation

- Use event delegation for many similar elements.
- Show loading indicators for slow operations.
- Use asynchronous techniques such as `fetch()` for retrieving data without reloading page.
- Provide immediate visual feedback to users.